

DuckDB: Embedded Analytics At Parquet Speed

The strongest practical finding is not only raw speed. DuckDB combines SQL ergonomics, direct Parquet scanning, vectorized execution, and in-process deployment without a separate database server.

WHAT IT IS

An embedded analytical database that runs inside the application process and exposes APIs across .NET, Node.js, Python, R, C/C++, Go, Rust, Java, browser/WASM, and more.

HOW IT WORKS

The benchmark queries **orders.parquet** directly, projects only **Quantity**, **UnitPrice**, and **RegionId**, then performs a SQL **GROUP BY** in-process.

WHY FAST

DuckDB is built for OLAP scans and uses a columnar-vectorized execution engine, processing batches of values instead of row-by-row interpretation.

FINDING

In the current 1B-row results, DuckDB delivered the fastest recorded run via Node.js and stayed very close via .NET.

MEASURED IN THIS REPO

319M rows/sec

DuckDB via Node.js processed 1,000,000,000 rows in 3.131 seconds.

DuckDB via .NET processed the same file in 3.387 seconds at 295M rows/sec.

orders.parquet



DuckDB
parquet scan



Vectorized
GROUP BY



Region
revenue

The Alternative Engines: Different Strengths

The comparison is not only a race. Each technology occupies a different layer of the analytical stack: embedded SQL engine, server OLAP database, columnar foundation, or DataFrame query engine.

SERVER OLAP

ClickHouse

A high-performance, column-oriented SQL database for OLAP workloads over very large datasets.

- Best fit: persistent analytical service, concurrent users, operational database deployment.
- Repo setup: Docker container with the repository **data** folder mapped into ClickHouse.
- Benchmark caveat: separate container startup, data loading, and pure query timing.

COLUMNAR FOUNDATION

Apache Arrow

A multi-language toolbox and standardized in-memory columnar format for high-performance analytical systems.

- Best fit: native systems, interoperability, and low-level columnar processing.
- Repo setup: C++ aggregator uses Apache Arrow with Parquet support.
- Tradeoff: high control and speed, but more implementation effort than SQL.

DATAFRAME ENGINE

Polars

A Rust-based DataFrame library available from Python, R, and Node.js, with a vectorized query engine and lazy optimization.

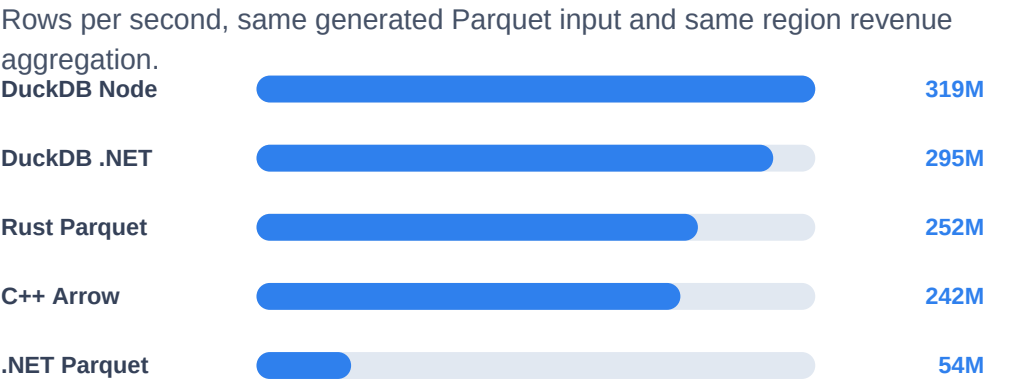
- Best fit: data engineering, data science, and expressive DataFrame workflows.
- Repo setup: Python aggregator uses Polars with PyArrow for Parquet processing.
- Tradeoff: excellent productivity, but API style differs from SQL-oriented tools.

What We Learned From The

Technology	Best Fit	Strength	Tradeoff
DuckDB	Local analytics over files	SQL, direct Parquet scan, very fast execution	Less suitable when a shared long-running service is required
ClickHouse	Production OLAP service	Scalable server-side analytics and operational DB features	Docker/server lifecycle and load path add complexity
Apache Arrow	Native columnar systems	Fast low-level columnar processing and interoperability	More code and engineering effort
Polars	DataFrame analytics	Productive API, lazy optimization, parallel execution	Performance depends on query shape and API usage

- DuckDB is the most impressive practical result: near-native scan speed with SQL ergonomics and no database server requirement.
- ClickHouse is the stronger choice when the problem becomes a shared analytical service instead of a local file-processing task.
- Arrow is the foundation for native columnar performance; Polars is the ergonomic DataFrame path.

1B-row throughput from Results.md



ClickHouse and Polars are described in the comparison, but are not included in this measured chart because matching 1B-row results were not captured in **Results.md**.